



Université Abdelmalek Essaâdi
Faculté des Sciences et
Techniques de Tanger



Rapport de projet

Adaptation de l'ANA au TSP

Version Edge-Memory ANA

Réalisé par : Youssef Khaloufi
Encadré par : Pr. Khalid Jebari

Année universitaire 2025–2026

Résumé

Ce rapport présente une adaptation de l'algorithme *Ant Nesting Algorithm* (ANA) au problème du voyageur de commerce, connu sous le nom de TSP. L'algorithme ANA original est une métaheuristique continue inspirée du comportement des fourmis *Leptothorax* lors de la construction d'un nid. Dans sa forme originale, chaque fourmi représente un agent de recherche dont la position est mise à jour à l'aide d'un poids de dépôt calculé à partir de la position courante, de la position précédente, de la meilleure position connue et des valeurs de fitness associées.

Le TSP est un problème discret basé sur des permutations. Une solution n'est pas un vecteur réel, mais un ordre de visite des villes. Pour cette raison, l'ANA original ne peut pas être appliqué directement. Le travail réalisé consiste donc à conserver l'esprit général de l'ANA, tout en remplaçant les opérations continues par des opérations adaptées aux tournées TSP.

La version développée, appelée **Edge-Memory ANA**, utilise une mémoire d'arêtes afin de guider la recherche vers les connexions qui apparaissent souvent dans les bonnes solutions. Elle introduit aussi deux traitements particuliers : une stratégie d'échappement lorsque l'agent est déjà sur la meilleure tournée globale, et une mini-reconstruction lorsque l'agent répète sa tournée précédente. Enfin, chaque nouvelle meilleure solution est améliorée par une recherche locale basée sur 2-opt et 3-opt.

Mots-clés : métaheuristiques, ANA, TSP, permutations, mémoire d'arêtes, 2-opt, 3-opt.

Table des matières

1	Introduction	1
1.1	Contexte général	1
1.2	Objectif du projet	1
1.3	Organisation du rapport	1
2	Résumé de l’algorithme ANA original	2
2.1	Inspiration naturelle	2
2.2	Entités de l’ANA	2
2.3	Modélisation générale	3
2.4	Cas particuliers de l’ANA	3
2.5	Mécanisme de fonctionnement	3
3	Le problème du voyageur de commerce	5
3.1	Définition	5
3.2	Représentation d’une solution	5
3.3	Pourquoi l’ANA original ne suffit pas	5
4	Adaptation proposée : Edge-Memory ANA	7
4.1	Idée générale	7
4.2	Mémoire d’arêtes	7
4.3	Classement des arêtes mémorisées	8
4.4	Mouvement positif	8
4.5	Mouvement négatif	8
4.6	Cas A : l’agent est sur la meilleure tournée globale	8
4.7	Cas B : la tournée courante est égale à la tournée précédente	9
4.8	Acceptation par recuit simulé	9

4.9	Amélioration locale	9
5	Mécanisme complet de la version finale	10
5.1	Pseudo-code	10
5.2	Ce qui a été changé par rapport à ANA	11
5.3	Paramètres principaux	11
6	Implémentation du projet	12
6.1	Choix de simplicité	12
6.2	Structure du code	12
6.3	Benchmarks utilisés	13
6.4	Critères de validation	13
7	Résultats et discussion	14
7.1	Objectif de l'évaluation	14
7.2	Résultats finaux sur les benchmarks retenus	14
7.3	Progression expérimentale de la méthode	15
7.4	Analyse des résultats	15
7.5	Limites	16
8	Conclusion	17

Chapitre 1

Introduction

1.1 Contexte général

Les problèmes d’optimisation apparaissent dans de nombreux domaines : transport, planification, ordonnancement, conception de réseaux, logistique et ingénierie. Dans ces problèmes, l’objectif est de choisir la meilleure solution possible selon un critère donné, par exemple minimiser une distance, un coût ou un temps.

Lorsque l’espace de recherche devient très grand, l’exploration exhaustive devient irréaliste. Les métaheuristiques constituent alors une famille de méthodes utiles. Elles ne garantissent pas toujours l’optimum global, mais elles permettent souvent de trouver de bonnes solutions dans un temps raisonnable.

1.2 Objectif du projet

L’objectif de ce projet est d’adapter l’algorithme ANA au problème du voyageur de commerce. Le but n’est pas de construire une bibliothèque générale de métaheuristiques, mais de produire une version claire, simple et fonctionnelle de l’ANA pour un problème discret basé sur des permutations.

Le travail suit trois idées principales :

- comprendre le mécanisme de l’ANA original ;
- identifier ce qui ne peut pas être utilisé directement pour le TSP ;
- proposer des opérateurs discrets équivalents, tout en conservant l’esprit de l’algorithme.

1.3 Organisation du rapport

Le rapport est organisé comme suit. Le chapitre 2 rappelle l’algorithme ANA original. Le chapitre 3 présente le TSP. Le chapitre 4 explique l’adaptation proposée. Le chapitre 5 décrit le fonctionnement de la version Edge-Memory ANA. Le chapitre 6 présente l’organisation du code et le protocole expérimental. Le dernier chapitre conclut le travail.

Chapitre 2

Résumé de l'algorithme ANA original

2.1 Inspiration naturelle

L'algorithme ANA est inspiré du comportement des fourmis *Leptothorax* pendant la construction d'un nid. Les fourmis ouvrières collectent des grains et les déposent autour de la reine et du couvain. Progressivement, les zones où les dépôts sont les plus importants deviennent plus attractives, ce qui conduit à la formation d'un mur autour du nid.

Dans l'algorithme, cette idée est transformée en mécanisme d'optimisation. Chaque fourmi artificielle représente une solution candidate. Les déplacements des fourmis correspondent à l'exploration de nouvelles solutions. La meilleure position connue joue le rôle de zone attractive.

2.2 Entités de l'ANA

Comme dans l'article original, les éléments naturels sont associés à des éléments algorithmiques. Le tableau suivant résume cette correspondance.

Élément naturel	Élément algorithmique
Fourmi ouvrière	Agent de recherche
Position de dépôt	Solution candidate
Qualité d'une position	Fonction objectif / fitness
Meilleure zone de dépôt	Meilleure solution trouvée
Ancienne position rencontrée	Position précédente de l'agent
Poids de dépôt	Intensité et direction du mouvement

2.3 Modélisation générale

Dans l'ANA original, une solution est représentée par une position continue $X_{t,i}$. À chaque itération, la position d'un agent est mise à jour selon :

$$X_{t+1,i} = X_{t,i} + \Delta X_{t+1,i} \quad (2.1)$$

où $\Delta X_{t+1,i}$ représente le changement de position. Dans le cas général, ce changement est calculé par :

$$\Delta X_{t+1,i} = dw \times (X_{best} - X_{t,i}) \quad (2.2)$$

Le terme dw est le poids de dépôt. Il dépend d'un nombre aléatoire r et d'un rapport entre deux tendances : la tendance courante T et la tendance précédente $T_{previous}$. L'idée est de comparer la situation actuelle de l'agent à sa situation précédente, en tenant compte à la fois de la distance à la meilleure solution et de la différence de fitness.

Pour un problème de minimisation, le principe peut être résumé par :

$$dw = r \times \frac{T}{T_{previous}} \quad (2.3)$$

Ce mécanisme permet de combiner exploration et exploitation :

- l'exploitation vient de l'attraction vers la meilleure solution ;
- l'exploration vient du facteur aléatoire r et du changement de direction possible.

2.4 Cas particuliers de l'ANA

L'ANA original distingue aussi deux situations particulières.

Premier cas : l'agent est déjà sur la meilleure solution. Dans ce cas, la direction vers la meilleure solution est nulle. L'algorithme utilise alors une forme de mouvement aléatoire pour éviter que l'agent reste bloqué.

Deuxième cas : la position courante est égale à la position précédente. Dans cette situation, l'agent ne possède pas de vraie information de mouvement. L'algorithme force alors une nouvelle direction vers la meilleure solution.

Ces deux cas sont importants dans notre adaptation, car ils correspondent aussi à des blocages possibles dans un espace de permutations.

2.5 Mécanisme de fonctionnement

Le fonctionnement général de l'ANA original peut être résumé simplement :

```
Initialiser une population de fourmis artificielles.
Initialiser la position précédente de chaque fourmi.
Tant que le nombre d'itérations n'est pas atteint :
    Trouver la meilleure solution actuelle.
    Pour chaque fourmi :
        Générer un nombre aléatoire r.
        Si la fourmi est sur la meilleure position :
            appliquer le cas spécial A.
        Sinon si la position courante = position précédente :
            appliquer le cas spécial B.
        Sinon :
            calculer T, Tprevious, dw et le déplacement.
        Générer une nouvelle solution.
        Accepter la solution si elle améliore la fitness.
Fin.
```

Chapitre 3

Le problème du voyageur de commerce

3.1 Définition

Le problème du voyageur de commerce, ou TSP, consiste à trouver la tournée la plus courte qui visite chaque ville exactement une seule fois et revient à la ville de départ.

Formellement, pour un ensemble de villes $V = \{0, 1, \dots, n-1\}$ et une matrice de distances $d(i, j)$, on cherche une permutation π qui minimise :

$$F(\pi) = \sum_{k=0}^{n-1} d(\pi_k, \pi_{k+1}) \quad (3.1)$$

avec $\pi_n = \pi_0$.

3.2 Représentation d'une solution

Dans ce projet, une solution est représentée par une permutation. Par exemple :

[0, 3, 2, 5, 1, 4]

signifie que la tournée commence à la ville 0, puis visite 3, 2, 5, 1, 4, avant de revenir à 0.

Pour éviter plusieurs représentations équivalentes d'une même tournée, la ville 0 est toujours placée en première position. Après chaque opération, la tournée est normalisée afin de conserver cette règle.

3.3 Pourquoi l'ANA original ne suffit pas

L'ANA original manipule des positions continues. Une mise à jour comme :

$$X_{t+1} = X_t + \Delta X \quad (3.2)$$

est naturelle lorsque X est un vecteur réel. Mais pour le TSP, une solution doit rester une permutation valide. On ne peut pas additionner directement un vecteur à une tournée.

Le défi principal du projet est donc le suivant :

Comment garder la logique de l'ANA tout en remplaçant le mouvement continu par des opérateurs valides sur des permutations ?

Chapitre 4

Adaptation proposée : Edge-Memory ANA

4.1 Idée générale

La version proposée conserve la structure de l'ANA : population d'agents, meilleure solution globale, position précédente, mouvement positif, mouvement négatif et cas particuliers. La différence principale est que les positions continues sont remplacées par des tournées TSP.

Le projet introduit une mémoire d'arêtes. Une arête correspond à une connexion entre deux villes. Dans le TSP, la qualité d'une tournée dépend fortement des arêtes qu'elle contient. Il est donc plus naturel de guider la recherche par les arêtes que par les positions des villes dans la permutation.

4.2 Mémoire d'arêtes

La mémoire d'arêtes est une matrice symétrique M de taille $n \times n$. Chaque valeur M_{ij} indique l'intérêt appris pour l'arête non orientée (i, j) .

Au départ, toutes les arêtes ont une valeur de mémoire identique. À chaque itération :

- la mémoire s'évapore légèrement ;
- les arêtes de la meilleure tournée globale reçoivent un dépôt ;
- les arêtes des solutions acceptées proches de la meilleure peuvent aussi recevoir un dépôt plus faible.

L'évaporation est donnée par :

$$M_{ij} \leftarrow (1 - \lambda)M_{ij} \tag{4.1}$$

où λ est le taux d'évaporation.

Lorsqu'une bonne tournée contient l'arête (i, j) , on applique :

$$M_{ij} \leftarrow \min(M_{ij} + \delta, M_{max}) \quad (4.2)$$

Cette mémoire joue un rôle proche de l'expérience collective de la population.

4.3 Classement des arêtes mémorisées

Pour éviter de parcourir toutes les arêtes à chaque mouvement, les arêtes sont classées dans un cache. Ce cache est marqué comme *dirty* lorsqu'une évaporation ou un dépôt modifie la mémoire. Il est reconstruit seulement quand il est nécessaire.

Le score d'une arête peut être pondéré par la distance :

$$score(i, j) = \frac{M_{ij}}{d(i, j)} \quad (4.3)$$

Ainsi, une arête est intéressante si elle possède une forte mémoire et une distance courte.

4.4 Mouvement positif

Dans l'ANA original, un mouvement positif rapproche l'agent de la meilleure solution. Dans notre version, ce principe est traduit ainsi :

Un mouvement positif essaie d'insérer dans la tournée courante des arêtes fortement mémorisées qui ne sont pas encore présentes.

Pour insérer une arête (a, b) dans une tournée, l'algorithme utilise une inversion de segment. Cette opération conserve toujours une permutation valide.

4.5 Mouvement négatif

Lorsque le poids de dépôt est négatif, l'algorithme effectue une inversion aléatoire d'un segment de la tournée. Ce mouvement joue le rôle d'exploration. Il permet de quitter une zone de recherche sans casser la validité de la permutation.

4.6 Cas A : l'agent est sur la meilleure tournée globale

Dans l'ANA original, lorsque l'agent est déjà sur la meilleure position, la direction vers la meilleure solution est nulle. Dans notre version, cette situation signifie que la tournée courante est exactement la meilleure tournée globale.

Pour éviter le blocage, la version Edge-Memory ANA applique un échappement adaptatif :

- si la stagnation est faible, l'algorithme tente d'insérer une arête mémorisée ;
- si la stagnation augmente, il tente plus d'insertions ;
- si aucune insertion utile n'est possible, il applique une inversion aléatoire.

4.7 Cas B : la tournée courante est égale à la tournée précédente

Lorsque la tournée courante est identique à la tournée précédente, l'agent n'a pas réellement progressé. Dans ce cas, la version proposée utilise une mini-reconstruction.

La mini-reconstruction insère plusieurs arêtes mémorisées dans la tournée courante. Si elle échoue, l'algorithme utilise une petite inversion aléatoire. Cette opération donne une nouvelle direction à l'agent.

4.8 Acceptation par recuit simulé

L'ANA original accepte principalement les améliorations. Dans notre version, une règle de recuit simulé est ajoutée pour permettre parfois l'acceptation d'une solution moins bonne. Cela aide à éviter certains optima locaux.

Si une solution candidate est meilleure, elle est acceptée. Sinon, elle peut être acceptée avec la probabilité :

$$p = \exp\left(-\frac{\Delta f/f}{T}\right) \quad (4.4)$$

où Δf est la détérioration de la fitness, f la fitness courante, et T la température courante.

4.9 Amélioration locale

Chaque fois qu'une nouvelle meilleure tournée globale est trouvée, elle est améliorée par une recherche locale :

1. appliquer 2-opt jusqu'à ce qu'aucune amélioration ne soit possible ;
2. essayer une amélioration 3-opt stricte ;
3. si 3-opt améliore, relancer 2-opt ;
4. répéter jusqu'à stabilisation.

Cette étape est importante, car l'ANA construit de bonnes directions globales, tandis que 2-opt et 3-opt corrigent localement la structure de la tournée.

Chapitre 5

Mécanisme complet de la version finale

5.1 Pseudo-code

Le fonctionnement de la version finale peut être résumé ainsi :

```
Initialiser la population de tournées.  
Calculer la fitness de chaque tournée.  
Déterminer la meilleure tournée globale.  
Initialiser la mémoire d'arêtes.  
Déposer les arêtes de la meilleure tournée dans la mémoire.
```

Pour chaque itération :

```
    Evaporer la mémoire d'arêtes.  
    Redéposer les arêtes de la meilleure tournée globale.  
    Calculer la température courante.
```

Pour chaque agent :

```
    Si la tournée courante est la meilleure globale :  
        appliquer le cas A : échappement adaptatif.  
    Sinon si la tournée courante = tournée précédente :  
        appliquer le cas B : mini-reconstruction.  
    Sinon :  
        calculer le mouvement ANA discret.  
        si  $dw > 0$  : insérer des arêtes mémorisées.  
        si  $dw < 0$  : appliquer une inversion de segment.
```

```
    Calculer la fitness de la tournée candidate.  
    Accepter si elle améliore la tournée courante.  
    Sinon accepter parfois avec le recuit simulé.
```

```
    Si la candidate est proche de la meilleure :  
        déposer ses arêtes dans la mémoire.
```

Si la candidate améliore la meilleure globale :
 appliquer 2-opt + 3-opt.
 mettre à jour la meilleure globale.
 déposer ses arêtes dans la mémoire.

Retourner la meilleure tournée trouvée.

5.2 Ce qui a été changé par rapport à ANA

Le tableau suivant résume les changements principaux.

ANA original	Version Edge-Memory ANA
Position continue X	Tournée TSP représentée par une permutation
Addition d'un déplacement ΔX	Opérateurs discrets : inversion et insertion d'arêtes
Direction vers X_{best}	Guidage par les arêtes mémorisées
Distance continue entre positions	Distance de similarité indirecte via swaps et structure d'arêtes
Cas agent = meilleur	Échappement adaptatif par mémoire puis inversion
Cas courant = précédent	Mini-reconstruction par arêtes mémorisées
Acceptation stricte des améliorations	Acceptation avec recuit simulé pour certaines détériorations
Pas de recherche locale spécifique TSP	Polissage 2-opt + 3-opt sur chaque nouvelle meilleure solution

5.3 Paramètres principaux

Les paramètres les plus importants sont :

- la taille de population ;
- le nombre maximal d'itérations ;
- le paramètre ρ , utilisé pour contrôler l'amplitude de certains mouvements ;
- le taux d'évaporation de la mémoire ;
- le dépôt sur la meilleure solution globale ;
- le dépôt sur les solutions proches de la meilleure ;
- la température initiale et minimale du recuit simulé.

Chapitre 6

Implémentation du projet

6.1 Choix de simplicité

Le projet final n'est pas une grande plateforme logicielle. Il s'agit d'une implémentation simple et lisible de la version finale de l'algorithme. L'objectif est de rendre le code compréhensible, en séparant seulement les responsabilités essentielles.

6.2 Structure du code

La structure finale du projet est la suivante :

```
edge_memory_ana_tsp/  
  main.py  
  core/  
    algorithm_flow.py  
    operators.py  
    local_search.py  
    tsp_utils.py  
  experiments/  
    benchmarks.py  
    runner.py  
  reporting/  
    results_writer.py  
  data/  
    wi29.tsp  
    dj38.tsp  
  results/  
  tests/  
    smoke_test.py
```

Le fichier le plus important est :

`core/algorithm_flow.py`

Il contient le flux général de l'algorithme : initialisation, boucle d'itérations, génération des candidats, acceptation, mise à jour de la mémoire et retour du résultat final.

Les fichiers de statistiques, d'écriture CSV et de résumé sont volontairement placés dans le dossier `reporting/`, afin de ne pas mélanger la logique de l'algorithme avec la présentation des résultats.

6.3 Benchmarks utilisés

La version finale utilise uniquement les benchmarks suivants :

Benchmark	Type	Optimum connu
square_4	synthétique	4.0
rectangle_6	synthétique	10.0
grid_9	synthétique	9.414213562373095
wi29	TSPLIB	27603
dj38	TSPLIB	6656

Pour les instances TSPLIB, les distances euclidiennes sont arrondies selon la convention `EUC_2D`.

6.4 Critères de validation

Une exécution est considérée comme réussie lorsque la meilleure fitness trouvée est égale à l'optimum connu. L'écart relatif est calculé par :

$$Gap(\%) = 100 \times \frac{f_{best} - f_{opt}}{f_{opt}} \quad (6.1)$$

Un écart de 0% signifie que l'optimum connu a été atteint.

Chapitre 7

Résultats et discussion

7.1 Objectif de l'évaluation

L'évaluation expérimentale ne cherche pas à prouver que cette adaptation dépasse les solveurs spécialisés du TSP. L'objectif est plus raisonnable : vérifier que l'algorithme final est correct, stable sur les instances retenues et capable d'atteindre les optima connus avec les paramètres fixés dans le projet.

Les résultats présentés ci-dessous sont donc des résultats de synthèse. Les lignes individuelles de chaque exécution ne sont pas nécessaires dans le rapport : elles sont conservées dans les fichiers CSV générés par le code, tandis que le rapport présente uniquement les indicateurs utiles pour juger la qualité de la version finale.

7.2 Résultats finaux sur les benchmarks retenus

Le tableau suivant présente les résultats retenus pour la version finale **Edge-Memory ANA**. Pour chaque benchmark, on indique le nombre d'exécutions, l'optimum connu, les valeurs obtenues et le nombre de succès.

Benchmark	Runs	Optimum	Meilleur	Moyenne	Succès	Temps moyen
square_4	10	4.0	4.0	4.0	10/10	0.028 s
rectangle_6	10	10.0	10.0	10.0	10/10	0.099 s
grid_9	30	9.4142	9.4142	9.4142	30/30	1.061 s
wi29	10	27603	27603	27603	10/10	37.74 s
dj38	10	6656	6656	6656	10/10	38.96 s

TABLE 7.1 – Résultats finaux retenus pour la version Edge-Memory ANA.

Ces résultats montrent que la version finale atteint l'optimum connu sur tous les benchmarks gardés dans le projet. Les trois petits benchmarks synthétiques servent surtout à vérifier la validité de l'implémentation : permutation correcte, ville 0 fixée, coût de tournée correct et comportement cohérent. Les deux instances TSPLIB, **wi29** et **dj38**, confirment que l'algorithme fonctionne aussi sur des instances réelles avec distances

entières.

7.3 Progression expérimentale de la méthode

Pendant le développement, plusieurs versions ont été testées. Le tableau suivant ne présente pas toutes les exécutions, mais uniquement les étapes utiles pour comprendre pourquoi la version finale a été retenue.

Version	Idée principale	Runs	Meilleur	Moyenne	Succès
V5	Adaptation discrète avec polissage 2-opt et 3-opt sur les nouvelles meilleures solutions.	50	27603	28127.7	8/50
V9	Ajout de la mémoire d'arêtes et du guidage par arêtes fortement mémorisées.	50	27603	27744.12	33/50
V11	Version finale : mémoire d'arêtes, cas A adaptatif et cas B par mini-reconstruction.	10	27603	27603	10/10

TABLE 7.2 – Résumé de la progression expérimentale sur l'instance wi29.

Cette progression montre l'intérêt de la mémoire d'arêtes. Le passage de V5 à V9 améliore fortement la robustesse sur **wi29** : le nombre de succès augmente de 8/50 à 33/50 et la moyenne se rapproche de l'optimum. La version V11 ajoute ensuite les deux traitements de blocage inspirés des cas particuliers de l'ANA : l'échappement adaptatif lorsque l'agent est déjà sur la meilleure tournée, et la mini-reconstruction lorsque la tournée courante répète la tournée précédente.

Il faut interpréter ce tableau comme une synthèse de développement, pas comme une étude statistique complète. Les nombres d'exécutions ne sont pas identiques entre toutes les lignes. Son rôle est de montrer la logique d'amélioration qui a conduit à la version finale.

7.4 Analyse des résultats

Les résultats obtenus confirment trois points importants.

Premièrement, la représentation par permutation est correcte. Toutes les tournées retournées restent valides : chaque ville apparaît une seule fois et la ville 0 est conservée en première position après normalisation.

Deuxièmement, la mémoire d'arêtes est adaptée au TSP. Dans un problème de tournée, la qualité dépend fortement des connexions entre villes. Guider les mouvements par des arêtes apprises est donc plus naturel qu'un déplacement positionnel direct.

Troisièmement, le polissage 2-opt/3-opt complète bien la recherche globale. L'ANA discret explore et propose de nouvelles tournées, puis la recherche locale corrige les croisements et améliore les détails de la tournée. Cette combinaison explique la capacité de la version finale à atteindre les optima retenus.

7.5 Limites

Ces résultats montrent que le projet est fonctionnel et cohérent, mais ils ne doivent pas être surinterprétés. Le TSP possède des solveurs spécialisés beaucoup plus puissants. L'objectif ici est d'étudier une adaptation de l'ANA au TSP, pas de remplacer des méthodes exactes ou des heuristiques très spécialisées comme Lin-Kernighan.

La principale limite concerne la montée en taille. Lorsque le nombre de villes augmente, la recherche locale 3-opt devient coûteuse. Une suite possible serait donc d'optimiser le temps de calcul, de réduire le coût de la recherche locale ou de comparer plus formellement la méthode à ACO, GA, SA et d'autres métaheuristiques classiques.

Chapitre 8

Conclusion

Ce projet a permis d'adapter l'algorithme ANA, initialement conçu pour des problèmes continus, au problème discret du voyageur de commerce. L'adaptation repose sur une idée simple : remplacer le mouvement continu par des opérateurs valides sur les permutations.

La version finale, Edge-Memory ANA, conserve les principes essentiels de l'ANA : population d'agents, meilleure solution, position précédente, mouvement guidé et cas particuliers. Elle ajoute une mémoire d'arêtes, une acceptation par recuit simulé et un polissage local 2-opt/3-opt.

Le résultat final est une implémentation simple, lisible et fonctionnelle. Le projet montre comment une métaheuristique continue peut être transformée en une méthode discrète, à condition de respecter la structure du problème traité.

Perspectives

Des améliorations possibles existent, notamment :

- comparer plus formellement l'algorithme à des méthodes classiques comme ACO, GA ou SA ;
- tester d'autres instances TSPLIB ;
- étudier l'influence des paramètres de mémoire ;
- améliorer la rapidité de la recherche locale.

Cependant, dans le cadre de ce projet, l'objectif principal était de construire une version claire, propre et compréhensible de l'ANA pour le TSP. Cet objectif est atteint.

Bibliographie

- [1] D. N. Hama Rashid, T. A. Rashid, S. Mirjalili, *ANA : Ant Nesting Algorithm for Optimizing Real-World Problems*, Mathematics, 2021.
- [2] G. Reinelt, *TSPLIB - A Traveling Salesman Problem Library*, ORSA Journal on Computing, 1991.
- [3] G. A. Croes, *A Method for Solving Traveling-Salesman Problems*, Operations Research, 1958.